# Monitoring Internal Covariant Shifts With Adjustable Standardized Weights

Charles de Sully
Johns Hopkins Whiting School of Engineering
cdesull1@jhu.edu

***Abstract*— Finding concept data drift is an ongoing process that poses a problem with data streams where known labeled truths are not readily available. With neural networks, unsupervised solutions tackle the issue of showing variance changes in input data. This research poses an alternative solution to showing variances within the neural network architecture by assuming each layer standardizes outputs to the following layers and tracking how neural node weights would theoretically update to keep a proper signal. The results show varied successful identification of drift with varied synthetic data sets designed to test fast and slow drift changes.**

***Keywords—online, drift, feature, covariate, standardize***

## I. INTRODUCTION

Trying to understand and model data in a production-level environment can meet pitfalls where the data's behavior changes over a sequence of iterations and no longer has the same desired outcome. "At any given prediction failure, the question arises as to whether this failure is simply due to noise, or

whether it is indicative that the concept is beginning to drift [1]. If the failure is from noise, then the system needs to expect this and be more robust. If the concept of the data is changing then there needs to be a detection of when, a quick recovery, adjustment of hypothesis to the new context, and retention of earlier experience for when concepts reappear [2].

When it comes to data models drifting, there are several ways it can happen; suddenly, such as when external factors such as public opinion quickly change to support alternative ideas; gradually as when mechanical devices wear over time; and then when the data distribution is altered but the underlying model still applies is virtual concept drift [3]. Widmer and Kubat describe virtual concept drift as an incomplete model where the teacher may be to blame because of insufficient data, or flawed original focus, and the computer model is faulty. Trying to model an investment data stream could easily fit under virtual concept drift as features such as the price of an asset should always be representative of the asset but the meaning behind the price is not fully understood. The asset price can then be altered by supporting features not yet present in the model. Further complicating the process are the features that were successfully contributing to the prediction are no longer relevant producing feature drift [4].

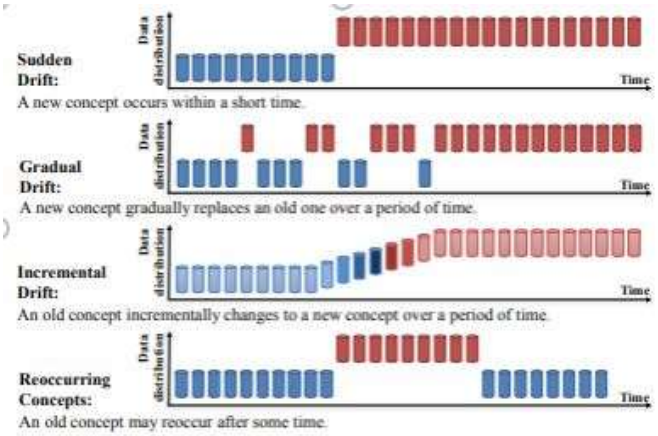



Fig. 1. Image source: https://arxiv.org/pdf/2004.05785.pdf

Understanding how a model can become irrelevant with some part of data drift gives way to the next step of how to detect when it happens so then choices can be made of what to do about it to make it relevant again.

One challenge for concept drift detection is being able to recognize true drift versus an outlier or noise coming from a random deviation or anomaly [5]. If an anomaly truly is present, then retraining need not occur.

Another challenge is trying to detect drift with classification performance, especially for streaming data which would need in-time online labeling and would be impractical in real-world situations as labeling is time-consuming and costly [6].

If expecting output classification promptly is impractical, then analysis can be done on the inputs. Domain drift, or covariate shift, is the term to name when the output distribution stays unchanged but the input signals' distribution in the model starts to vary [7] [8]. Statistical tests, or some other type of method, would then be done on the inputs to ensure consistency.

This paper will take the examination of domain drift internally into the individual nodes of a neural network and supply some empirical evidence that it can be successful to show when covariate shifting happens.

## II. Related Work

Adaptive Windowing (ADWIN) Based on sliding windows analyzing data through set time periods, drift is detected when the mean of two sub-windows exceeds a threshold difference. Window sizes are variable depending on data variances [9].

Drift Detection Method (DDM) Based on the availability of labels the prediction error rate is checked with a binomial distribution. Assuming the distribution is stationary, added examples should show a decrease in error rate. When the error rate rises, the model has drifted [10].

Early Drift Detection Method (EDDM) Instead of the number of errors to track, this method compares the distance of two errors when appropriately trained, and the distance between two errors when sampling in production. The drift happens when the distance between two errors comes closer together. Drift signals get produced when the distance crosses below a defined threshold [11].

Micro-Cluster Nearest Neighbor (MS-NN) This online method proposes clusters where the statistics are kept on which cluster receives a misclassified prediction. Clusters with a good prediction get added to, clusters with a bad prediction get marked down as well as the cluster that the bad prediction should have belonged to. Once a cluster receives enough misclassified counts it is split to better account for current variations in data. This split rate as well as deprecated clusters removal rate are tracked to interpret as drift occurrences [12].

Uncertainty Drift Detection (UDD) This method describes taking an aggregate prediction set and the variance is used as a record point for ADWIN. If ADWIN detects a large enough variance change then drift is detected. This allows for avoiding needing a label that ADWIN would normally need and using a statistical online approach in place of the label [13].

## III. Approach

### A. Background

An artificial neural network (ANN) is typically made up of a neuron model, inspired by brain cells, which could be supplied with many input signals, process them and then output that signal to another forward neuron. [14]. The first half of the neuron behaves like a linear equation where input magnitudes are changed, summed up, and then a bias quantity is added on. What differentiates an artificial neuron from a linear equation is the second half where the linear output is then sent through a nonlinear equation. Typical nonlinear equations are of the sigmoid form where large negative values are asymptote to a minimum boundary and large positive values are asymptote to an upper boundary. Other nonlinear equations exist that enhance learning behavior depending on the architecture of the network [15].

Neural Networks in literature normally arrange neurons in groups or layers. Each layer processes signals from an earlier layer and feeds the modified signals forward to the next layer. The expectation is that each layer gets one step closer to the predictive goal. The last layer is then read by a system to interpret the result.

For the ANN to learn an error quantity is formulated from the output compared to the expected result. That error is then fed back through the neural nodes, layer by layer, in a process called backpropagation. The error is resized depending on the nonlinear slope it exists on at the time it was output. A derivative of the nonlinear equation is used to adjust this error. Feeding the error backward then is magnified by the weights of the neural node to give a proportion of responsibility to the weight that may have caused the error [16].

Learning problems exist where backpropagation is not the most effective process. Other researchers have come up with alternatives, such as RPROP and its derivatives. These alternatives overcome an issue called the vanishing gradient where error magnitudes get smaller and smaller as the error is fed further and further back into the layers. This promotes little weight changes further back in the network stalling the learning. RPROP and others ignore magnitudes and look for sign changes in the value to figure out adjustments [17].

### B. Going Forward

Another enhancement was found to standardize the input values going in the model [18]. Lecun, et al., report faster convergence giving an extreme example when all input is positive then the weights can only increase or decrease together slowing the training down by forcing the system to zigzag across errors. Standardizing the data and moving the mean to zero becomes more efficient. More research has built upon this by standardizing the inputs to each internal node in hidden layers as well [19].

It makes sense that when looking at a domain shift, standardizing input data to have a mean of zero and a deviation of one would be the step to take to understand when data starts to become more than one standard deviation in behavior [20]. This same approach can be used on the internal covariates.

In this research, the learning mechanism of backpropagation will actively be using standard methods to get to a trained state. Once trained, the training data will be used to set up an internal standardized state of outputs. This will be done by adding a shadow mechanism to the neural node outputs that will track the mean and variance in an online rolling fashion to accommodate scenarios of streaming data. The reason for a focus on streaming data is that the problem state is harder. Streaming data does not have the luxury of having immediate labels if predictions are wrong, and batch learning is more difficult with the quantity and velocity of data [21]. The approach of using a shadow mechanism is to allow the neural network to behave as expected and to virtualize a what-if scenario of a standardized data stream flowing between hidden layers.

As a virtual standardized output is produced it will not be compatible with existing forward weights awaiting the value. Shadow weights will be made using the real weights as a starting point and the adjusted mean and variance will dictate the change. This will allow the original signal value from the earlier sigmoid output to reach the next sigmoid input the same. This can be done easily because the mechanism is linear. It will also need to be known that while a shadow weight will be adjusted once per sample value, the bias will need adjustment by each node output from the earlier layer.

State 1: neural node output x to next layer node:

$$c + \alpha(x)$$

State 2:

Given a rolling average and variance of node output x:

$$\mu = \lambda x + (1 - \lambda)\mu_{-1}$$

$$\sigma^2 = \lambda(x - \mu)^2 + (1 - \lambda)\sigma^2_{-1}$$

Then standardized x:

$$\hat{x} = \frac{(x - \mu)}{\sqrt{\sigma^2}}$$

$$c + \alpha(x) \neq c + \alpha(\hat{x})$$

To counter the standardizing adjustments so the forward activation function keeps value integrity:

The forward weight must first cancel the deviation:

$$c + \sigma\alpha\left(\frac{(x - \mu)}{\sigma}\right)$$

Then the mean value must be cancelled out:

$$c + \alpha(\mathrm{x}) - \alpha\mu + \alpha\mu$$

Each forward weight will keep the adjustment:

$$\alpha_1 = \sigma\alpha_0$$

Each forward bias will progressively update its value for each back node:

$$c_1 = c_0 + \sum_{w=0}^{n} \alpha_w \mu_w$$

$$c_0 + \alpha_0(x) = c_1 + \alpha_1(\hat{x})$$

Because the bias weight is a conglomeration of adjustments by an entire earlier layer of nodes, it will be used as a dimensionality reducer to give representation to a node versus the substantial number of other weights the node could have if earlier layers are enormous in size.

Added monitoring mechanisms will be added to find the standardized range that a bias shadow weight will be in flux through. The expectation is that as covariate data shifts, the resting range of the shadow weights will shift also.

It can be considered that checking a neural node's output alone could be enough to find drift. The motivation behind the research focusing on the bias shadow weight is because the bias is capturing the interaction of an entire layer of nodes' behaviors. Some signals may cancel out another, but also there could be a large additive effect enhancing drift capture.

## IV. Experiments

The experiments will be using synthetic data "Synthetic datasets for concept drift detection purposes" [22]. There are four types of data sets produced by five different signal generators. In each set, there is an abrupt drift and a gradual drift. Some sets have noise added. The author used the scikit-multiflow framework using the following generators: sine, random tree, mixed, sea, and stagger. The datasets have 40000 examples with a single output class of 1 or 0.

The abrupt drifts occur at steps 10000, 20000, and 30000. The gradual drifts occur at steps 9500, 20000, and 30500 where the width of the drift is 1000 steps.

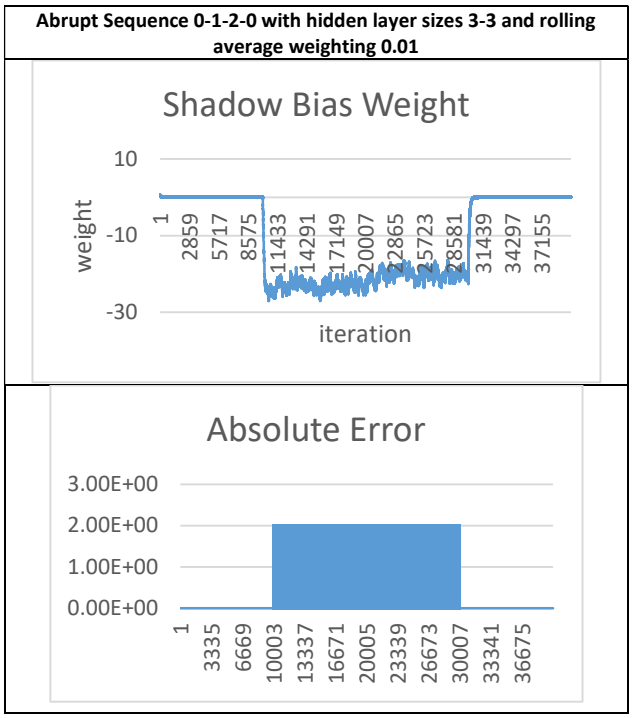
The neural network will be an RPROP variant with varying layers depending on the complexity of the data set.

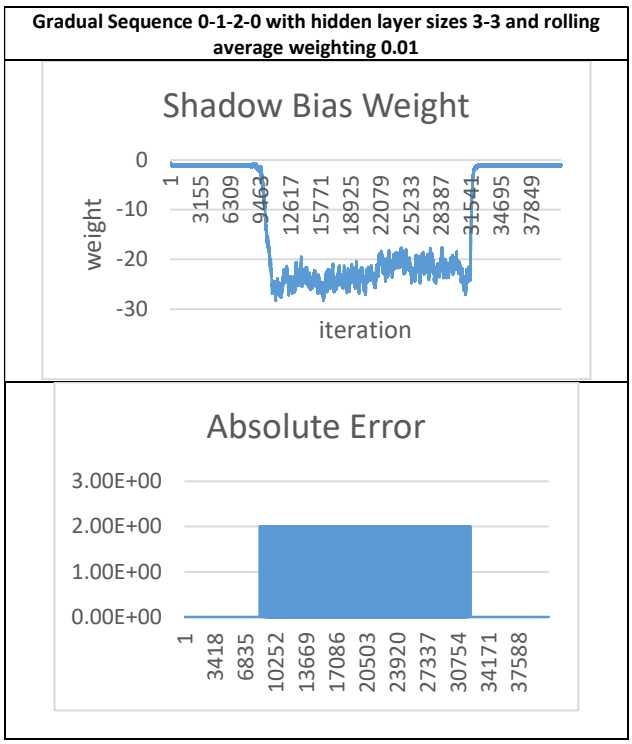
The first 9000 examples will be provided as a training set for three epochs in batch update sizes of 100 with random sampling. Next, the weights will be frozen, then the entire dataset will feed forward through the neural network. The shadow weights that adjust according to the standardized signals will be the only moving parts being tracked. To reiterate, the shadow weights adjust to offset the standardized value to reform to their original value as if standardization had not happened.

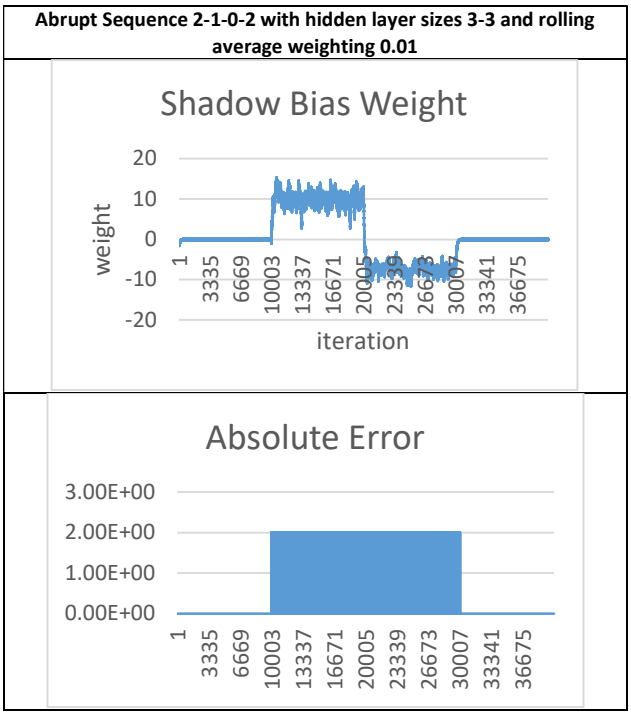
## V. Results

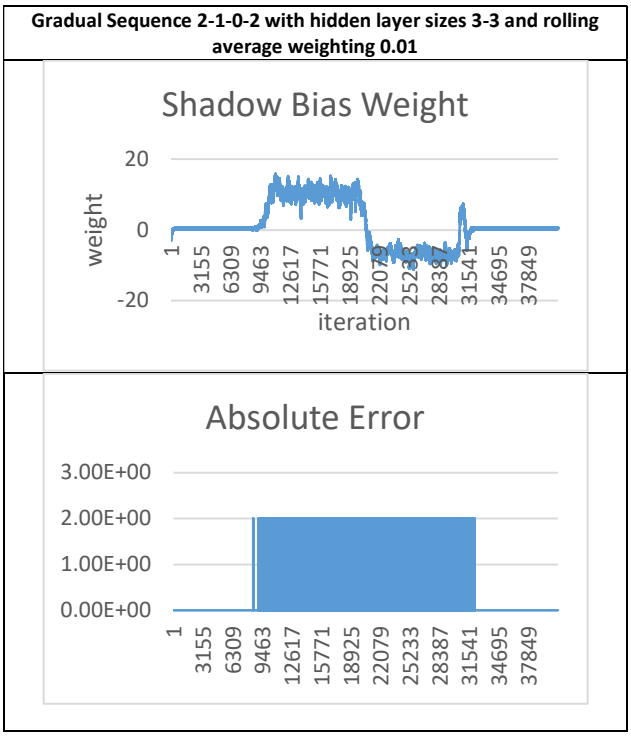
Stagger Generator:

A sequence of classification functions, three numerical features, no noise.



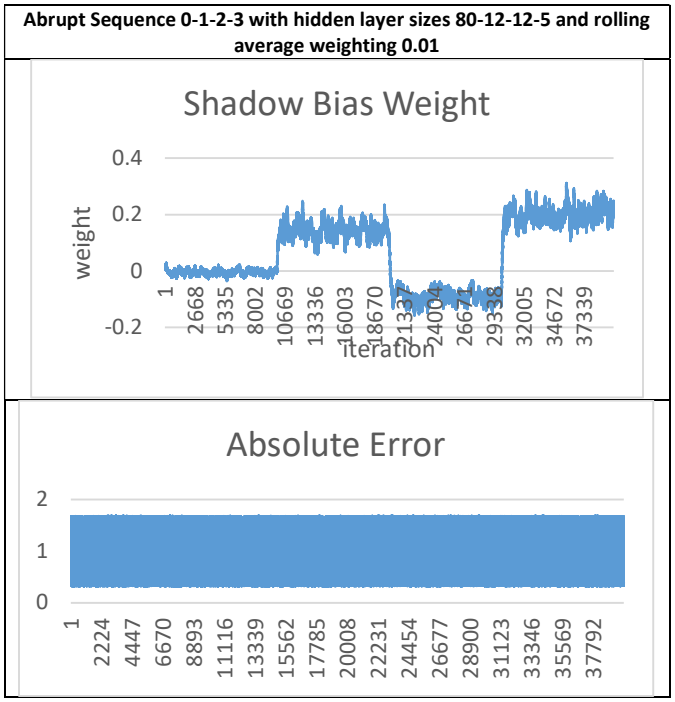



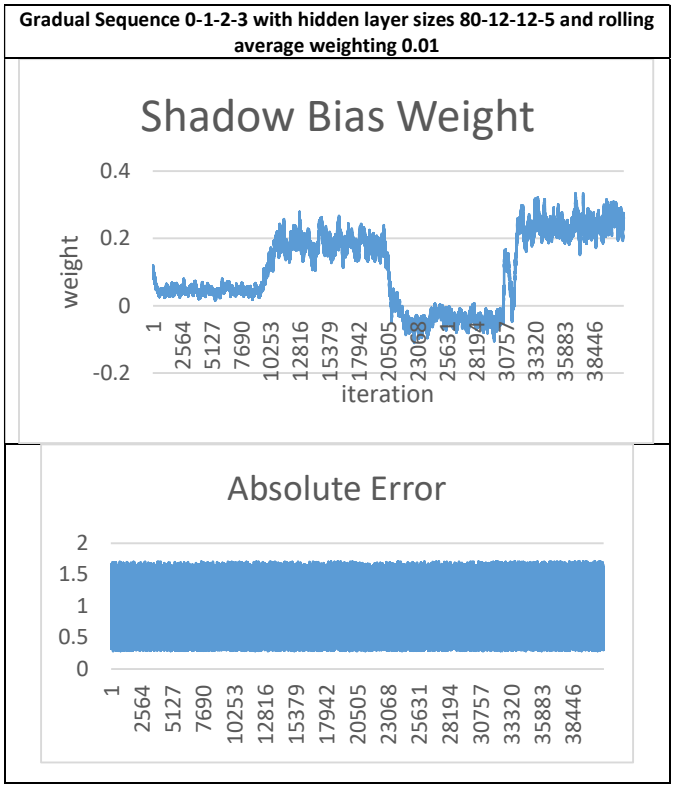



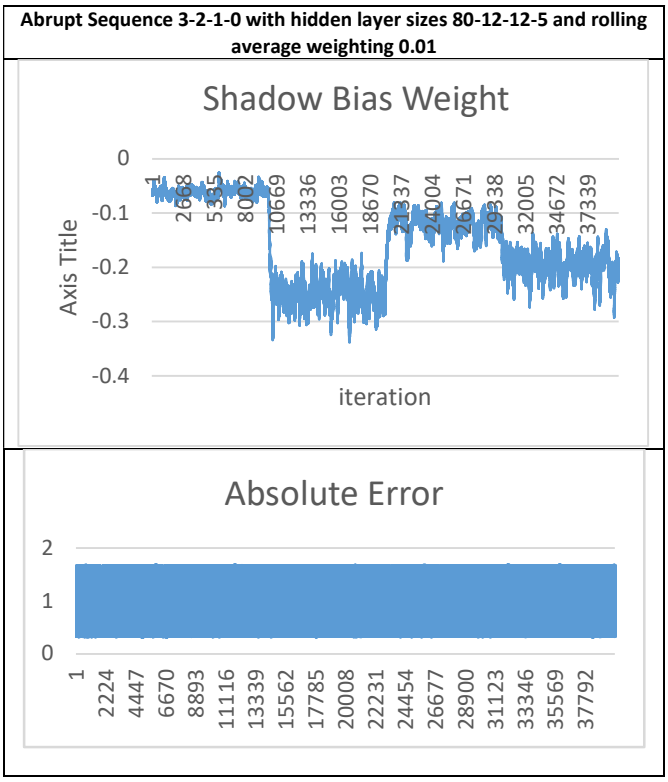

The stagger data set can quickly be found by shadow weights when drift occurs with a minimal neural architecture. The 0-1-2-0 sequence does not distinguish well between the transition from 1 to 2. The 2-1-0-2 sequence does have a clear distinction between drift sets. The categorical error is also in line with what the neural net was trained with and confirms the shadow weight updates.

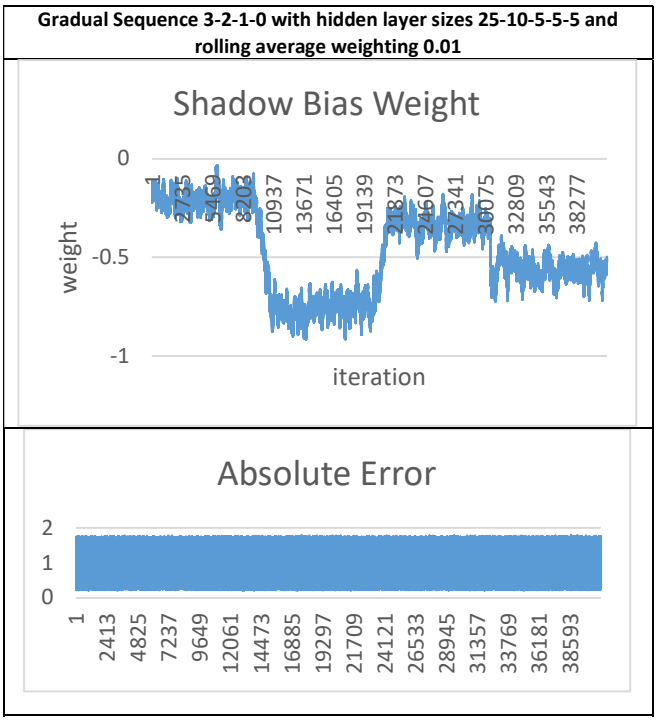
Sea Generator:

A sequence of classification functions, three numerical features, noise probability 20%.



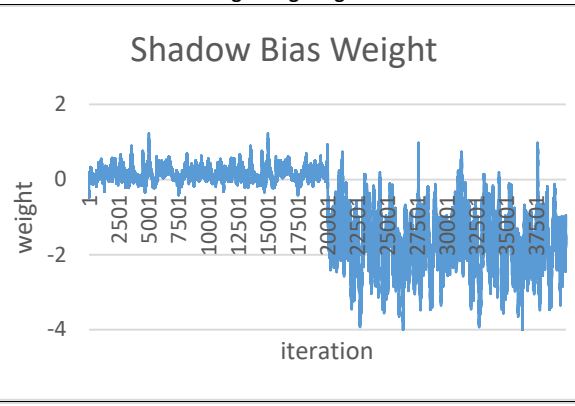



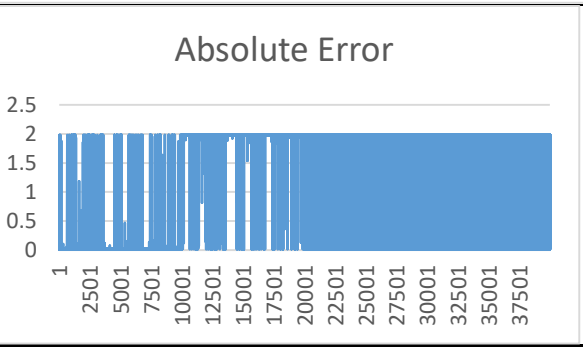

The Sea generator is very uniform with categorical errors on the neural network output making it quite interesting how the shadow bias can pick up the drift even through the noise.

Sine Generator:

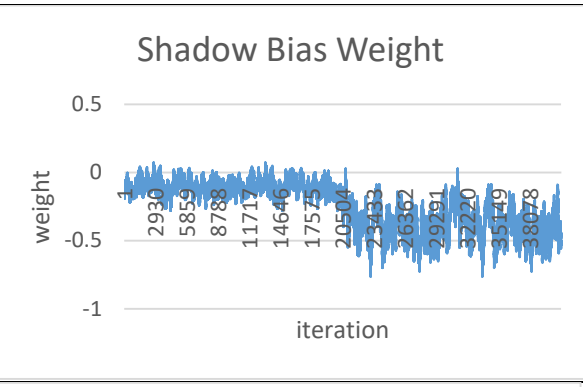
A sequence of classification sine functions, two numerical features, no noise.

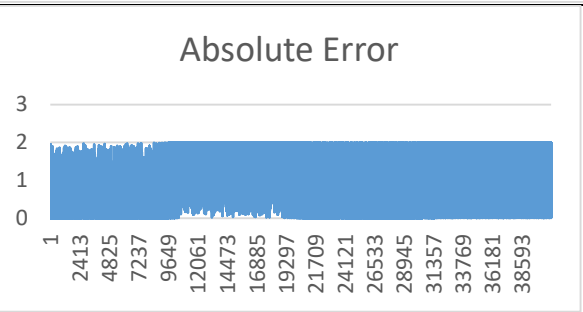
**Abrupt Sequence 0-1-2-3 with hidden layer sizes 80-12-12-5 and rolling average weighting 0.01**



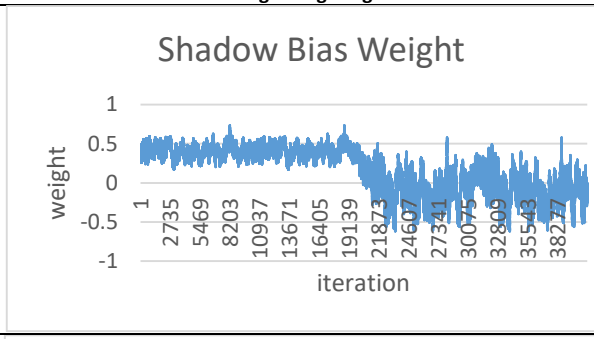



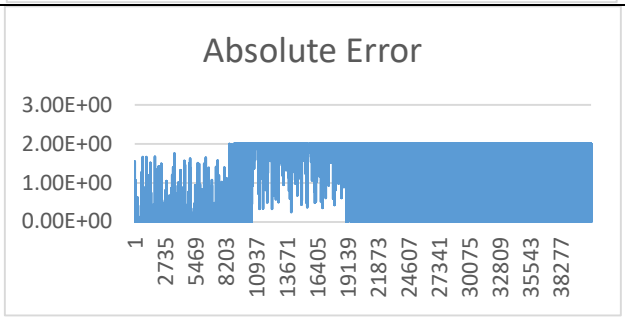

**Gradual Sequence 3-2-1-0 with hidden layer sizes 80-12-12-5 and rolling average weighting 0.01**



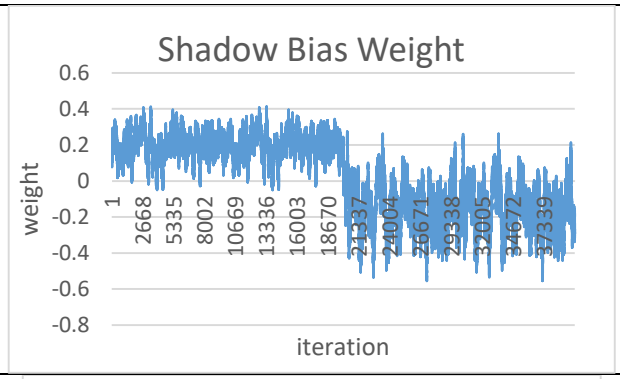



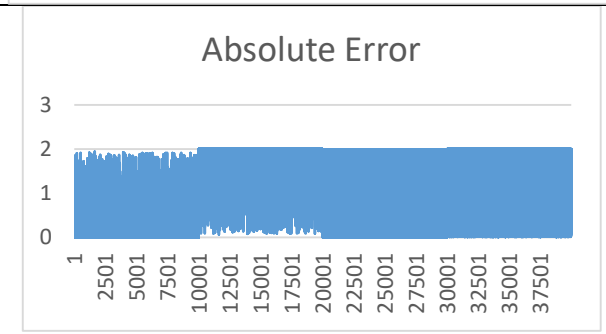

**Gradual Sequence 0-1-2-3 with hidden layer sizes 80-12-12-5 and rolling average weighting 0.01**




**Abrupt Sequence 3-2-1-0 with hidden layer sizes 80-12-12-5 and rolling average weighting 0.01**

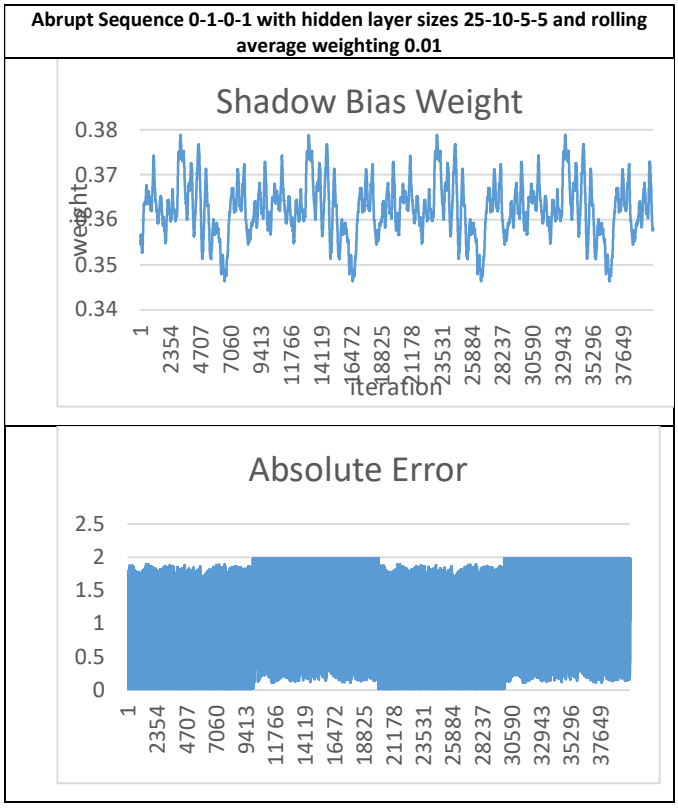


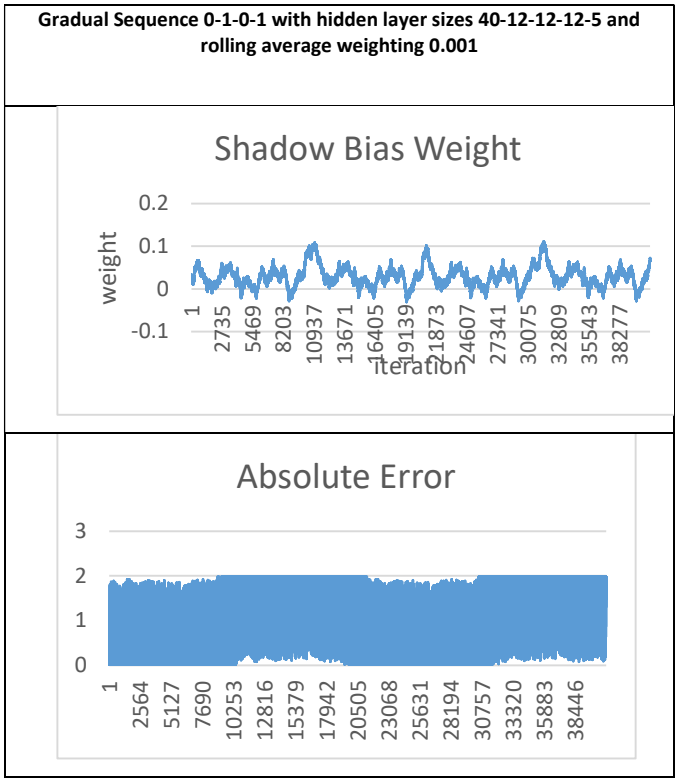



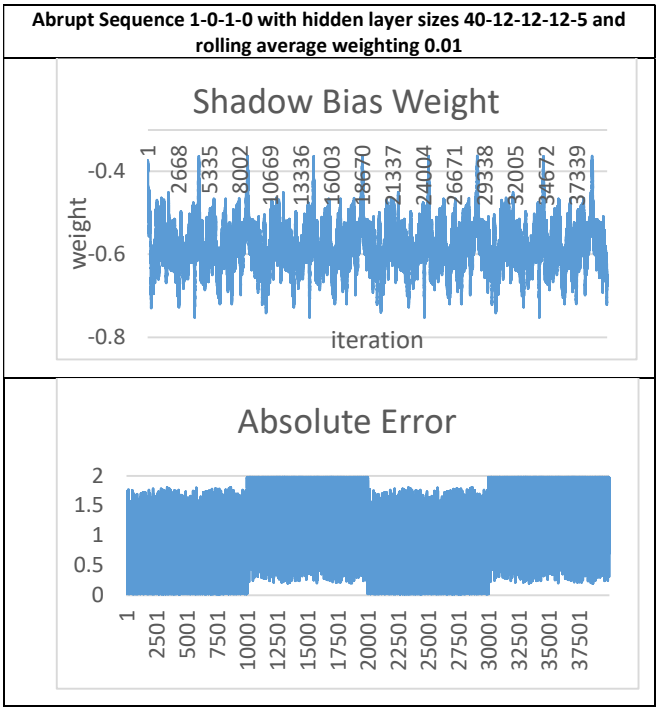

The sine generator is making it difficult for the neural network to find the first data drift. Categorical error shows the drift where it should be.

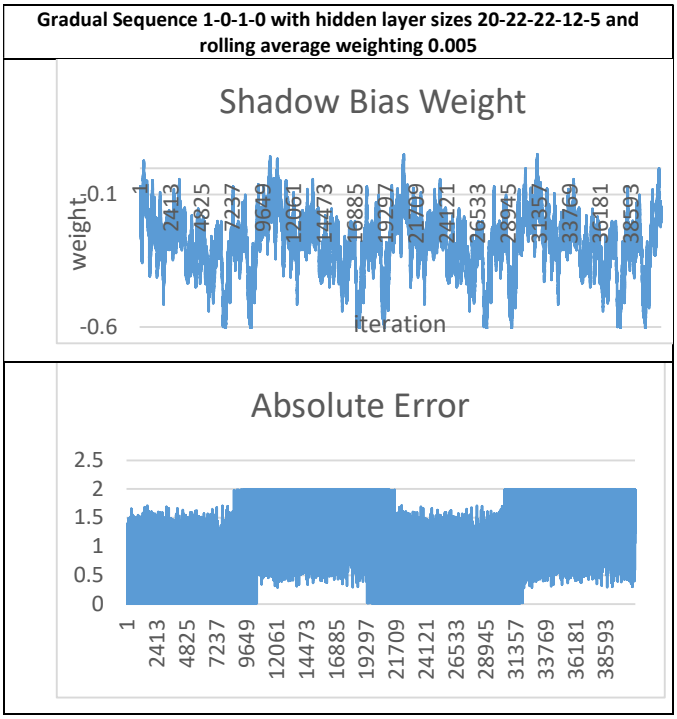
Mixed Generator:

A sequence of classification functions, four numerical features, no noise.



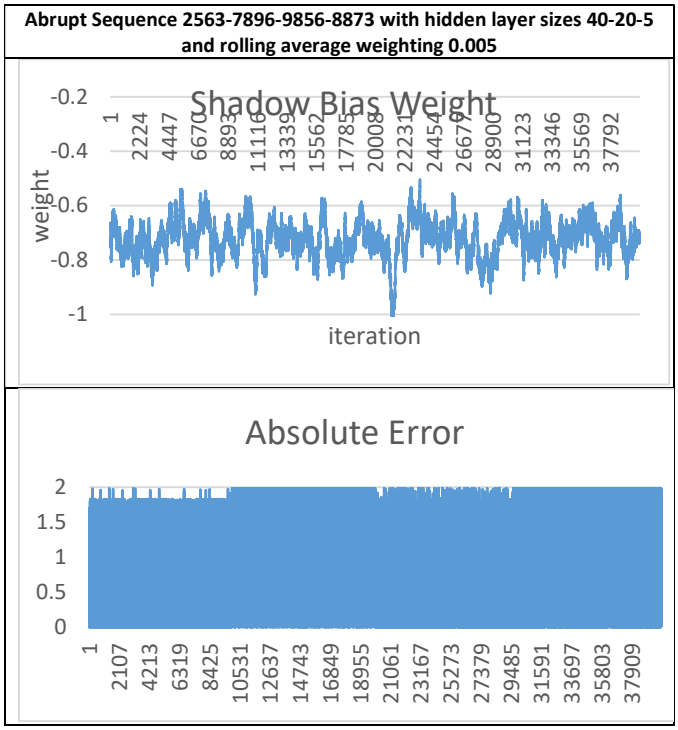



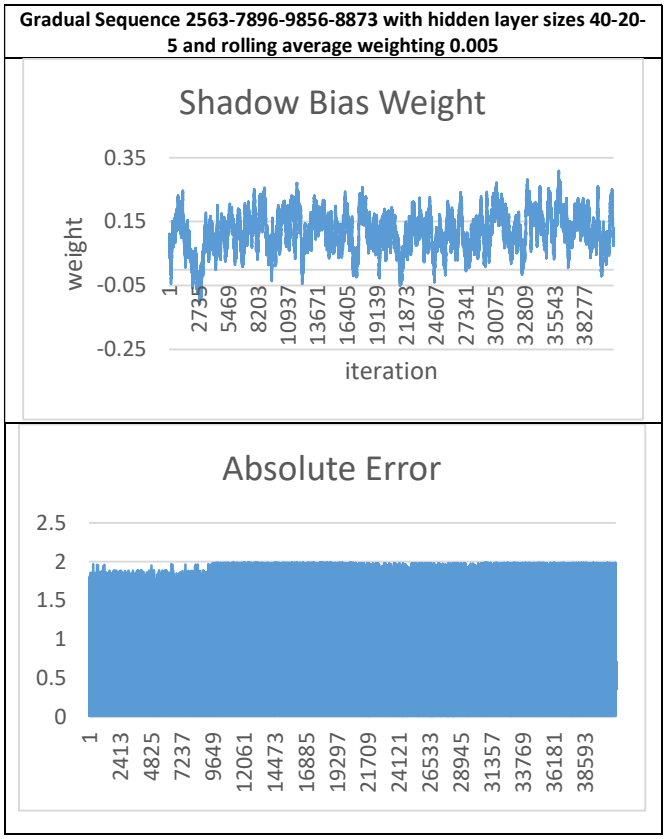



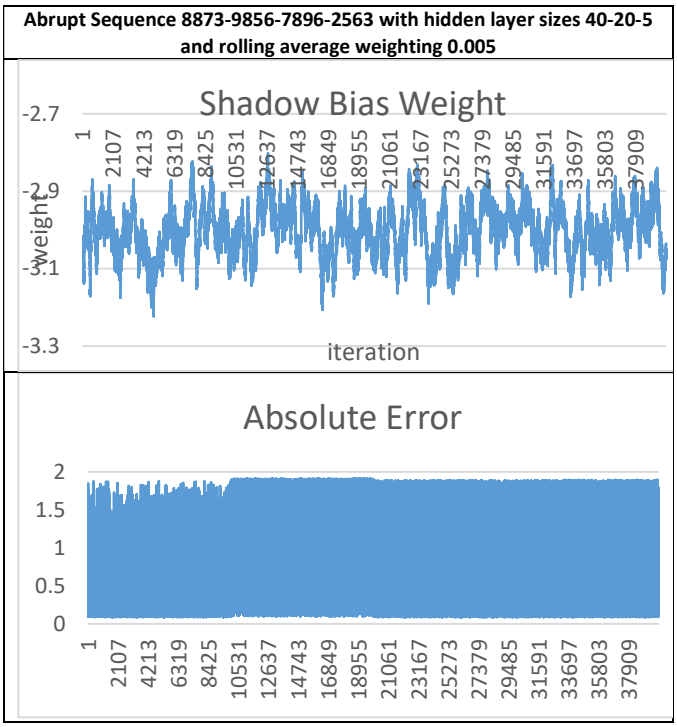

The mixed generator produces clear categorical error boundaries on the neural network output, but the covariant shift has a different cycle, not fully in line with the output error. Whatever the internal drift may be, the distribution is not revealing.

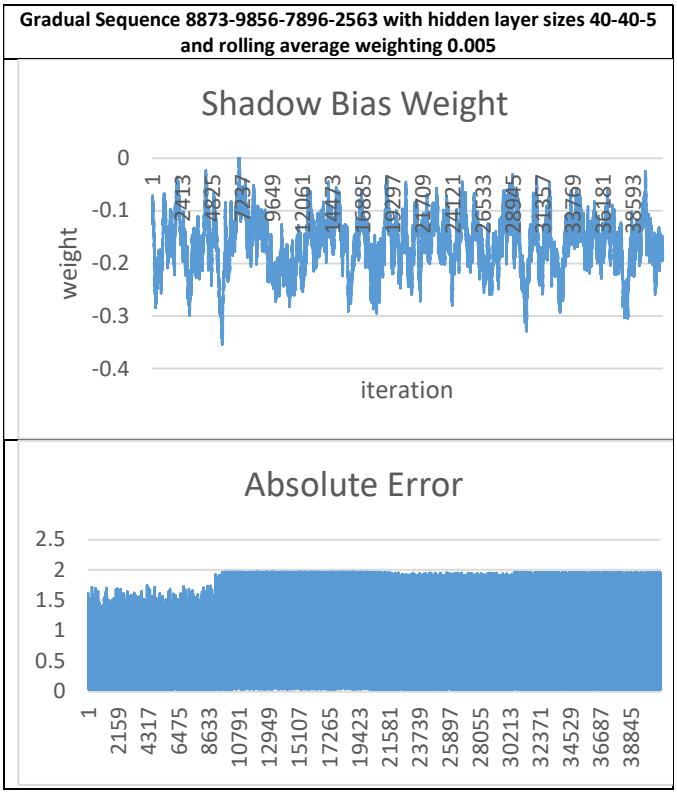
Random Tree Generator:

A sequence of tree random state functions, two numerical features, no noise.




The random tree generator set proves difficult to uncover drift. The output error has barely enough variation to show drift boundaries. The shadow weight has no distinguishable pattern as if random. The bias weight has frequency variation that may make these results candidates for a frequency transform to find the covariant drift.

## VI. Conclusion

Attempting to solve concept data drift with streaming data in neural networks by detecting internal covariant shifts with adaptable standardized weights proved to have problem sets to overcome. The highlights were from the Stagger data set and the Sea data set with noise where clear boundaries were seen. The Mixed and Random Tree data sets have qualities that elude the internal standardization techniques, while Sine had partial success. More research is needed to discover whether a frequency domain should be analyzed or some other method in the related works.

With the research, it was found that drift became clearer the further in the layers the signal went. The current idea is the first few layers are delegated to filtering noise out so the data structure may not be fully plain. As the layers continue the concept structure gets formulated and disturbances to that structure are more readily seen.

Future work could go along similar lines as the Uncertainty Drift Detection method and use the variance in the standardized weights to feed into ADWIN.

Other work will be needed to figure out if non-bias standardized weights can direct fault to earlier layer nodes responsible for feature drift.

As a final thought, it can be considered that training examples that have yet to be trained on could present the same covariance shift signals as testing data because the training data has not yet been seen. Building on that idea, any node showing data drift may need to be trained more thoroughly or have added neural nodes brought in to reduce the variance workload. This could allow targeting particular neural nodes to update weights and avoid costly backpropagation on an entire network with an exceptionally large node count.

## References


[1] [1] J. C. Schlimmer and R. H. Granger, "Incremental Learning from Noisy Data," Kluwer Academic Publishers, 1986.

[2] [2] G. Widmer, "Learning in the Presence of Concept Drift and Hidden Contexts," 1996.

[3] [3] G. Widmer and M. Kubat, "Effective Learning in Dynamic Environments by Explicit Context Tracking."

[4] [4] J. P. Barddal, H. M. Gomes, F. Enembreck, and B. Pfahringer, "A survey on feature drift adaptation: Definition, benchmark, challenges and future directions," *Journal of Systems and Software*, vol. 127, pp. 278–294, 2017, doi: https://doi.org/10.1016/j.jss.2016.07.005.

[5] [5] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys (CSUR)*, vol. 46, pp. 1–37, 2014.

[6] [6] Y. Xu and D. Klabjan, "Concept Drift and Covariate Shift Detection Ensemble with Lagged Labels," in *2021 IEEE International Conference on Big Data (Big Data)*, 2021, pp. 1504–1513. doi: 10.1109/BigData52589.2021.9671279.

[7] [7] H. Shimodaira, "Improving predictive inference under covariate shift by weighting the log-likelihood function," *J Stat Plan Inference*, vol. 90, no. 2, pp. 227–244, 2000.

[8] [8] J. Quinonero-Candela, M. Sugiyama, A. Schwaighofer, and N. D. Lawrence, *Dataset shift in machine learning*. Mit Press, 2008.

[9] [9] A. Bifet and R. Gavalda, "Learning from time-changing data with adaptive windowing," in *Proceedings of the 2007 SIAM international conference on data mining*, 2007, pp. 443–448.

[10] [10] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Brazilian symposium on artificial intelligence*, 2004, pp. 286–295.

[11] [11] M. Baena-Garca, J. del Campo-Ávila, R. Fidalgo, A. Bifet, R. Gavalda, and R. Morales-Bueno, "Early drift detection method," in *Fourth international workshop on knowledge discovery from data streams*, 2006, vol. 6, pp. 77–86.

[12] [12] M. Shakir, H. Supervisor, and F. Stahl, "Real-Time Pre-Processing Technique for Drift Detection, Feature Tracking, and Feature Selection using Adaptive Micro-Clusters for Data Stream Classification," 2018.

[13] [13] L. Baier, T. Schlör, J. Schöffer, and N. Kühl, "Detecting concept drift with neural network model uncertainty," *arXiv preprint arXiv:2107.01873*, 2021.

[14] [14] G. A. Carpenter, "Neural network models for pattern recognition and associative memory," *Neural networks*, vol. 2, no. 4, pp. 243–257, 1989.

[15] [15] G.-B. Huang and H. A. Babri, "Upper bounds on the number of hidden neurons in feedforward networks with arbitrary bounded nonlinear activation functions," *IEEE Trans Neural Netw*, vol. 9, no. 1, pp. 224–229, 1998.

[16] [16] J. Leonard and M. A. Kramer, "Improvement of the backpropagation algorithm for training neural networks," *Comput Chem Eng*, vol. 14, no. 3, pp. 337–341, 1990.

[17] [17] A. Mosca and G. D. Magoulas, "Adapting resilient propagation for deep learning," *arXiv preprint arXiv:1509.04612*, 2015.

[18] [18] G. B. Orr and K.-R. Müller, *Neural networks: tricks of the trade*. Springer, 1998.

[19] [19] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *International conference on machine learning*, 2015, pp. 448–456.

[20] [20] D. T. J. Huang, Y. S. Koh, G. Dobbie, and A. Bifet, "Drift detection using stream volatility," in *Joint European conference on machine learning and knowledge discovery in databases*, 2015, pp. 417–432.

[21] [21] B. Babcock, S. Babu, M. Datar, R. Motwani, and J. Widom, "Models and issues in data stream systems," in *Proceedings of the twenty-first ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*, 2002, pp. 1–16.

[22] [22] J. López Lobo, "Synthetic datasets for concept drift detection purposes," *Harv. Dataverse*, 2020.